

TORTOISESHELL / GOLD BROWLINE — STUDY 02

Build these glasses in Three.js

An annotated construction blueprint, a working 3D model, and editable curve data.

Reconstruct the mottled acetate brows and arms, thin gold eyewires, metal bridge, clear lenses and separate nose pads from the supplied reference.

SOURCE PHOTOGRAPH



SUPPLIED THREE.JS RECONSTRUCTION



The visible design can be matched closely. Exact product size, hidden geometry and lens prescription cannot be established from one angled photograph. The orthographic drawings and dimensions in this guide are explicit estimates.

TORTOISESHELL / GOLD BROWLINE BLUEPRINT

Separate acetate brows + gold eyewires + clear nose pads · shared curve data · dimensions are inferred

STUDY 02

UNITS: mm

01 FRONT ELEVATION

Two acetate brows · thin closed gold eyewires · separate metal bridge



REFERENCE / VISIBLE FEATURES

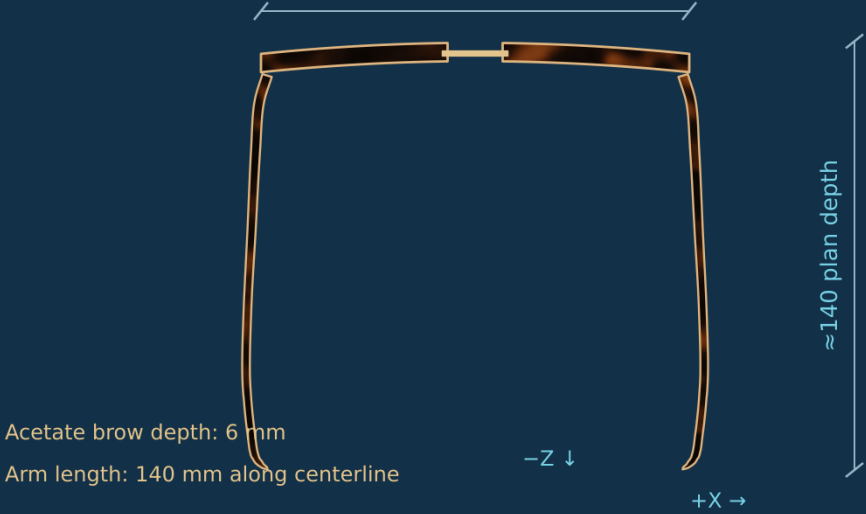
Appearance from one angled photograph; manufacturer dimensions unknown



- Amber / dark-brown mottling on brows and arms
- Gold bridge, thin eyewires and oval rivets
- Clear lenses; small nose pads on bent posts
- Pattern layout and hidden hardware are inferred.

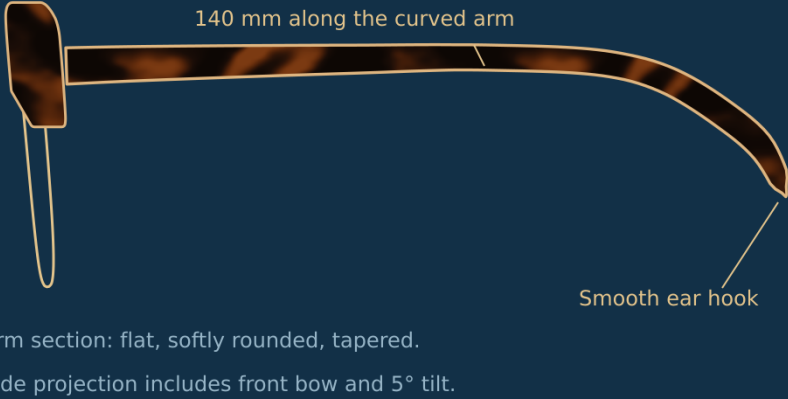
02 TOP / PLAN VIEW

Acetate depth and arm flare · bridge and pad supports in gold



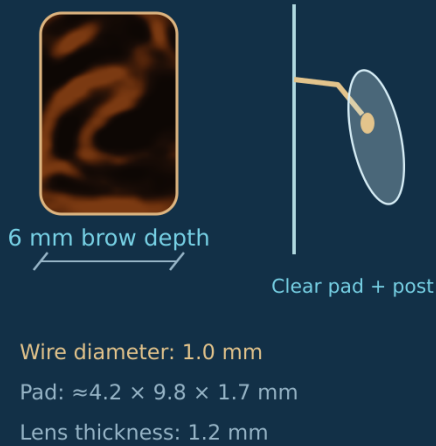
03 SIDE ELEVATION

Flat acetate shaft and downturned tip · thin gold front below brow



04 BROW / PAD DETAILS

Enlarged section sketches; inferred construction



COORDINATES

+X right · +Y up · +Z forward · bridge-plane origin · output units: meters

ASSEMBLY

2 brows + 2 gold eyewires + bridge + 2 lenses + 2 arms + 2 nose pads / posts

DESIGN ESTIMATE

One perspective photo. Orthographic views, dimensions, pattern placement and hidden details are inferred.

Source geometry: design.json + glasses-model.js / Three.js r186

Open the package

- Open browline-workbench.html directly in a WebGL-capable browser. The model, Three.js, reference, generated texture and blueprint are self-contained; no CDN is required.
- Drag to orbit, scroll to zoom and right-drag to pan. Front, Side and Top buttons use an orthographic camera.
- Adjust front width, arm length, roughness, arm opening, separation and gold wire diameter. Toggle lenses, nose pads and wireframe.
- New mottling changes the procedural pattern seed. Reset restores the supplied reconstruction.
- Export GLB produces an assembled, unfolded model with the texture embedded. It then restores the pose and visibility you were inspecting.
- tortoiseshell-browline.glb is ready to import. src/glasses-model.js, src/design.json and src/tortoiseshell-texture.js are the editable procedural source.
- browline-blueprint.svg is the vector drawing; the PNG is the high-resolution sheet. textures/tortoiseshell-basecolor.png contains the supplied pigment map.

1. Identify the visible construction

Visible feature	Modeling choice
Dark brown / amber upper rims	Two independent beveled acetate brow meshes
Thin gold-colored lower rims	Closed metal paths following each lens contour
Small gold bridge between the brows	Separate gently curved metal band
Clear lenses	Two gently bowed, thin physical lens solids
Clear oval nose pads	Two flattened transparent pad meshes with gold-colored supports
Mottled flat arms	Swept rounded rectangular sections with a pigment texture
Small oval front and side details	Independent flattened gold-colored ellipsoids

The curved lower lens edges are defined by the thin metal eyewires. The brows cover the upper portions and extend downward at their outside corners. The bridge is a separate metal connection; the reference has no continuous plastic bridge across the center.

Fit the silhouette before the pattern. The farther lens looks narrower in the angled photograph. Use a symmetric front as a starting assumption and reproduce the difference with camera perspective, front bow and the original curved contours.

2. Establish coordinates and estimated dimensions

The following values describe the default reconstruction, not manufacturer measurements. Changing the width slider scales the complete front in X, including lenses and their spacing.

Parameter	Starting value	Definition
Front width	140 mm	Outside-to-outside acetate brows
Front height	50.8 mm	Highest brow to lowest gold eyewire
Lens contour	52 × 43 mm	Bounding size of the construction contour
Lens gap	18 mm	Gap between contour bounding boxes
Acetate brow depth	6 mm	Local thickness including bevels
Brow bevel	0.48 mm	Rounding around the silhouette
Bevel depth	0.55 mm	Rounding toward front and rear surfaces
Gold eyewire diameter	1.0 mm	Round section; editable in the workbench
Lens thickness	1.2 mm	Approximate local optical solid
Arm centerline length	140 mm	Curved path length from hinge to tip
Arm section near hinge	3.2 × 6.5 mm	Thickness × height; tapers along the shaft
Nose pad local dimensions	4.2 × 9.8 × 1.7 mm	Before pad tilt and rotation
Front bow	3.6 mm	Side edges recede from the bridge plane
Front tilt	5°	Small estimated pantoscopic tilt

The 18 mm contour gap is not a confirmed marked bridge size. The modeled bridge band overlaps the brows and spans approximately 21.4 mm. The full assembled width is slightly greater than the front width because the arms flare outward.

Use +X to the right when viewed from the front, +Y upward, and +Z toward the viewer. The bridge plane is the origin. The arms extend toward −Z. Authoring values are millimeters; the returned geometry and transforms are meters. A 140 mm front is 0.140 scene units wide. Do not apply a second 0.001 scale after importing the supplied model.

3. Construct brows and thin gold eyewires



Define the right brow with the cubic Bézier commands in `browRight`. Its upper edge has a broad shallow arch; the inner end rounds down near the bridge, while the outside corner extends farther down. Mirror X for the other brow.

Extrude each brow separately. With a 6 mm target depth and 0.55 mm bevel depth on each face, the core extrusion is 4.9 mm. Use a small 0.48 mm silhouette bevel and `bevelOffset: -bevelSize` to retain the nominal contour size. `ExtrudeGeometry` provides the depth, curve and bevel controls [1].

```
const geometry = new THREE.ExtrudeGeometry(browShape, {
  depth: 4.9,
  curveSegments: 40,
  bevelEnabled: true,
  bevelSize: 0.48,
  bevelThickness: 0.55,
  bevelOffset: -0.48,
  bevelSegments: 4
});
geometry.translate(0, 0, -4.9 / 2);
```

Define the lens contour with `innerRight`. Normalize its bounds to 52 × 43 mm, place its inner X limit at +9 mm, and mirror it. Keep the lens contour independent of the acetate brow so you can tune the thin lower edge and the thick upper part separately.

Convert that 2D contour into a closed 3D curve. Apply the same front bow and tilt to the brows, eyewires and lenses:

```
z += -wrap * (x / (frontWidth / 2)) ** 2
  + y * Math.tan(THREE.MathUtils.degToRad(tilt));
```

Use `TubeGeometry` for the thin round metal eyewire [2]. The curve returns meter coordinates, so a 1 mm wire has a 0.0005 m radius.

```
const wire = new THREE.TubeGeometry(
  eyewireCurve, 240, 0.0005, 10, true
);
```

The upper metal path continues beneath the brow. Its lower arc remains visible. Check that the wire meets the brow at both ends and follows the lens edge without a visible gap. Slight hidden lens overlap prevents a bright seam. Preserve the rounded lower contour rather than substituting a circular ring.

After deformation, merge coincident vertices and recompute normals for smooth bevels. Generate acetate UV coordinates after this step; otherwise merging or removing UVs can erase the color-map information needed for the pattern and GLB export.

4. Build the bridge, lenses and nose pads



The bridge commands create a gently curved gold-colored band between the brow inner ends. Extrude it approximately 1.6 mm, bevel its edge by 0.15 mm, and offset it slightly forward. Its ends overlap the brow bodies so the connection appears continuous.

Reuse the lens contours for separate thin solids. Expand each around its center by 0.3% to hide the edge inside the gold eyewire. Subdivide the broad lens faces before adding a subtle bow; moving only the perimeter can leave large flat facets.

The source lens material uses physical transmission [3]:

```
const glass = new THREE.MeshPhysicalMaterial({
  color: 0xffffffff,
  roughness: 0.025,
  transmission: 1,
  opacity: 1,
  ior: 1.5,
  thickness: 0.0012
});
```

Keep opacity at 1 when using physical transmission. This is an appearance reconstruction of clear lenses, not a recovered optical prescription. The final reflections depend on the receiving application's environment and tone mapping.

For each nose pad, flatten a smooth sphere into a $4.2 \times 9.8 \times 1.7$ mm oval. Place its center near (± 9.9 , -4.0 , -7.3) mm and give it a slight mirrored tilt. Use a pale clear polymer appearance: roughness 0.18, transmission 0.78, IOR 1.42. The material composition and pad angle are inferred.

Connect each pad to the inner eyewire with a bent gold-colored post of approximately 0.66 mm diameter. Add the small pad attachment separately. The post should recede toward the face, while the clear cushion remains visible around its mount. The reference supports the visible arrangement; it does not reveal exact adjusters, screw threads or hinge hardware.

5. Sweep the arms and position the hinges



Use the eleven `templeControlPoints` to build a centripetal Catmull-Rom curve. The shaft is mostly straight with a mild outward flare, followed by a smooth downturned ear hook. Mirror X for the second arm.

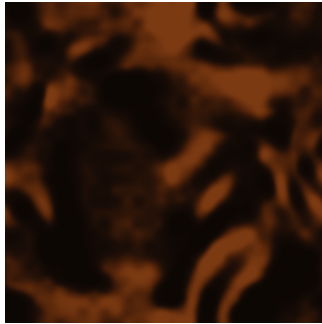
Normalize the centerline's arc length to 140 mm around its hinge position. The plan-view depth is shorter than the traveled curve length. This distinction matters when changing the ear-hook shape.

At each curve sample, sweep a flattened rounded rectangular section: approximately 3.2×6.5 mm at the hinge, tapering toward roughly 2.5×4.5 mm along the shaft. The supplied section uses a superellipse and a smoothly capped tip. Its UV coordinates run along the arm length so the pigment scale remains comparable to the brows.

Put each arm in a group at the hinge. `setOpen(90)` is fully open; smaller angles rotate the groups inward around opposite Y directions. The small gold-colored hinge pins are visual approximations. Mechanical limits and exact folded clearances are not established by the photograph.

Parent	Children
Model root	front_assembly; lens_L; lens_R; temple_pivot_L; temple_pivot_R
front_assembly	brows; gold eyewires; bridge; nose pads; posts; pad mounts; hinge pins
brow_L / brow_R	front_rivet_L / front_rivet_R
temple_pivot_L / temple_pivot_R	corresponding arm and side rivet

6. Recreate the tortoiseshell pattern



The source uses a deterministic 512×512 pigment map. It combines periodic noise at several scales, warps the domain into irregular patches and maps the result through dark chocolate and subdued amber colors. This approximates the reference's material character; the hidden pattern and exact placement of each visible patch cannot be recovered from the photograph.

`textureSeed` controls the patch layout. New mottling changes it; Reset restores seed 1836. The texture is a normal base-color map so it can be baked into GLB, rather than depending on a custom runtime shader.

`DataTexture` accepts the generated pixel buffer [4]. Because these pixels represent color, label the texture as `sRGB` [5]. Enable repeating and mipmaps to make the small mottles stable when viewed from a distance.

```
const texture = new THREE.DataTexture(
  data, 512, 512, THREE.RGBAFormat
);
texture.colorSpace = THREE.SRGBColorSpace;
texture.wrapS = texture.wrapT = THREE.RepeatWrapping;
texture.magFilter = THREE.LinearFilter;
texture.minFilter = THREE.LinearMipmapLinearFilter;
texture.generateMipmaps = true;
texture.needsUpdate = true;
```

Use the texture on the two brows and the two arms. Their base material color is white, so the map supplies the pigment colors. Metal, lenses and pads use separate untextured materials.

The brows use continuous projected UVs that include X, Y and a small Z contribution. The arms use longitudinal UVs. These are editable in `projectAcetateUvs()` and `templeUvs()`. Changing UV scale changes patch size; changing the seed changes the arrangement. The brow mapping spans approximately one texture repeat per 66 mm horizontally.

The included PNG contains the same source pixels as the default generated map. If loading that PNG manually to replace the `DataTexture`, use the same wrapping and `sRGB` setting, and set `flipY = false` to follow this source's UV convention. Using the exported GLB handles the baked texture and UVs together.

7. Match materials, lighting and the photograph

Part	Appearance	Metalness	Roughness	Other
Brows and arms	Dark chocolate / amber map	0	0.22	Clearcoat 1; clearcoat roughness 0.14
Gold-colored wire and hardware	#c6a36b	0.85	0.22	Small restrained highlights
Lenses	Clear	0	0.025	Transmission 1; IOR 1.5
Nose pads	Pale clear polymer	0	0.18	Transmission 0.78; IOR 1.42

Use an environment map to give the glossy plastic and thin metal something to reflect [3]. The workbench generates a studio-like environment with RoomEnvironment, an upper-left light and a faint ground shadow. Keep the amber patches restrained so the brow's outline remains readable.

The starting comparison camera is an estimate: position (0.170, 0.070, 0.190) m, target (0, -0.006, -0.049) m, nominal vertical FOV 38°. The viewer widens the vertical FOV on narrow screens to retain the whole model. Orthographic cameras are used for blueprint inspection.

Match camera yaw, elevation, perspective strength and framing before changing the brow geometry to imitate the angled image. Compare the two front rivets, brow inner tips, bridge, lowest eyewire points and ear-tip ends. Then adjust front bow, curve shape, pigment scale and reflection intensity.

Approximate reference landmarks from the 384 × 341 photograph:

Landmark	Approximate pixel (X, Y)
Far outer brow corner	(19, 110)
Far oval rivet	(26, 121)
Bridge middle	(140, 146)
Near oval rivet	(279, 162)
Near lower gold rim	(224, 239)
Near ear-tip end	(363, 167)

8. Integrate or export the model

The source is tested with Three.js 0.186.1. Copy all three source files into a project using a JSON-aware bundler: `glasses-model.js`, `design.json`, and `tortoiseshell-texture.js`.

```
import { createGlasses } from './src/glasses-model.js';

const glasses = createGlasses({
  frontWidth: 140,
  templeLength: 140,
  wireDiameter: 1.0,
  textureSeed: 1836
});
scene.add(glasses.group);
glasses.setOpen(90);
glasses.setExploded(0);
```

On removal, detach the group and call `glasses.dispose()`. It releases the geometries, materials and generated GPU texture. The source keeps only a small recent cache of pixel buffers for quick rebuilding.

To use the ready-made GLB:

```
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';
const { scene: asset } = await new GLTFLoader()
  .loadAsync('/models/tortoiseshell-browline.glb');
scene.add(asset); // already in meters
```

The workbench uses `GLTFExporter` to embed geometry, UVs, materials and the tortoiseshell image into one binary file [6]. Clearcoat and transmission use supported glTF extensions. Cameras, lights, floor, blueprint and source reference are excluded. Reflections may differ in another renderer because lighting and tone mapping belong to the receiving application.

The supplied export was re-imported with 21 meshes, four textured meshes and both nose pads preserved. The assembled bounds are approximately $152.3 \times 50.8 \times 140.1$ mm; the front itself remains 140 mm wide. The detailed reconstruction contains 55,642 triangles.

9. Refine toward a measured product match

1. Obtain straight-on front, top and side photographs with a ruler in the part's plane. Measure lens size, bridge, brow thickness, wire diameter and arm length.
2. Fit the brow and lens contours to the front view. Check the brow corner drops, the bridge connection and the thin lower rim.
3. Fit the front bow, arm flare and ear hook to the top and side views. Replace the approximate hinge and nose-pad hardware with measured details.
4. Capture higher-resolution front and side material photographs if the exact mottling matters. Replace or paint the pigment map and keep the UVs aligned across surfaces.
5. Match the supplied perspective image through camera calibration, then tune roughness and illumination. Validate the silhouette before judging reflections.

For mobile use, accept the silhouette before reducing curve, tube and bevel subdivisions. Cap pixel ratio, reduce shadow resolution and profile lens and pad transmission on the target device. The supplied model prioritizes inspection detail; the triangle count and physical transparency are not a mobile performance guarantee.

Sources and package notes

[1] Three.js ExtrudeGeometry: <https://threejs.org/docs/pages/ExtrudeGeometry.html>

[2] Three.js TubeGeometry: <https://threejs.org/docs/pages/TubeGeometry.html>

[3] Three.js MeshPhysicalMaterial: <https://threejs.org/docs/pages/MeshPhysicalMaterial.html>

[4] Three.js DataTexture: <https://threejs.org/docs/pages/DataTexture.html>

[5] Three.js Texture color management: <https://threejs.org/docs/pages/Texture.html>

[6] Three.js GLTFExporter: <https://threejs.org/docs/pages/GLTFExporter.html>

The user's uploaded `image(20261006-100052).png` is copied unchanged as `reference.png`. Geometry and pigment pixels are generated procedurally. The Three.js MIT license is included in the standalone workbench and in `THIRD-PARTY-LICENSES.txt`.